Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет: Компьютерных систем и сетей
Кафедра: Электронных вычислительных машин
Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
КОНВЕРТЕР БЕЛОРУССКИХ РУБЛЕЙ В ЕВРО»
Вариант 1

Студент: Т.М. Ананьева

Преподаватель: Д.В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить основные возможности интегрированной среды разработки Microsoft Visual Studio Code при создании объектно-ориентированного приложения на языке C++ с использованием библиотеки Qt 6, освоить сборку, запуск и отладку программы с графическим интерфейсом, а также применить архитектурный паттерн MVC с активной моделью.

В соответствии с вариантом 1 необходимо разработать приложение для конвертирования белорусских рублей в евро. Для выполнения работы поставлены следующие задачи:

1. Создать и настроить проект C++ (Qt Widgets) в Visual Studio Code, изучить средства редактирования, сборки, запуска и отладки.
2. Подобрать расширения Visual Studio Code для работы с UML и SysML v2 и отразить результат настройки в отчёте.
3. Разработать графическое приложение с главной формой и отдельным окном ввода суммы и курса.
4. Обеспечить запоминание последних введённых данных и их восстановление при повторном вводе.
5. Реализовать проверку входных данных и вывод понятного сообщения при ошибке.
6. Разделить приложение на модель, представление и контроллер и реализовать активное уведомление представления моделью.
7. Проверить основные сценарии работы программы и подготовить ответы на контрольные вопросы.

Работа выполняется в операционной системе Windows с использованием свободно распространяемых инструментов: среды разработки Visual Studio Code, компилятора MinGW (g++ 13.1), системы сборки CMake с генератором Ninja и библиотеки Qt 6.11.1 (модуль Widgets). Результатом работы должно стать приложение с графическим интерфейсом, исходный код которого разделён по паттерну MVC и не требует установки дополнительных библиотек, кроме поставляемых с Qt.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Интегрированная среда разработки и её компоненты

Интегрированная среда разработки (Integrated Development Environment, IDE) объединяет инструменты, необходимые для создания программы: редактор исходного кода, средства навигации и рефакторинга, систему сборки, средства запуска, отладчик, терминал, интеграцию с системой контроля версий и механизм расширений.

Visual Studio Code – расширяемая среда разработки, в которой перечисленные компоненты представлены отдельными областями и расширениями. Основные элементы окна: боковая панель Explorer с деревом проекта, редактор с вкладками файлов, панель Problems с ошибками и предупреждениями, терминал, консоль отладки Debug Console и панель управления системой контроля версий Source Control. Поддержка языка C++ добавляется набором расширений C/C++ Extension Pack, а сборка проекта выполняется расширением CMake Tools.

2.2 Редактирование и навигация в Visual Studio Code

Редактор Visual Studio Code поддерживает автодополнение (IntelliSense), подсказки типов, быстрые исправления, переход к определению и поиск использований. Основные приёмы работы: Ctrl+P – быстрый переход к файлу, Ctrl+Shift+P – палитра команд, Ctrl+Shift+F – поиск по проекту, F12 – переход к определению, Shift+F12 – поиск использований, F2 – безопасное переименование символа, Shift+Alt+F – форматирование документа, Ctrl+/ – комментарий строки. Встроенная проверка кода отображает ошибки и предупреждения в панели Problems, а структура текущего файла – в панели Outline.

2.3 Сборка, запуск и отладка приложения на C++ с библиотекой Qt

Исходные файлы C++ компилируются в машинный код. В работе используется компилятор MinGW (g++ 13.1), генератор сборки Ninja и система сборки CMake: файл CMakeLists.txt описывает проект, подключение библиотеки Qt 6 Widgets и список исходных файлов. Сборка выполняется командами

```
cmake -S . -B build -G Ninja -
DCMAKE_PREFIX_PATH=C:/Qt/6.11.1/mingw_64
cmake --build build
```

В Visual Studio Code эти команды оформлены задачей в файле .vscode/tasks.json и запускаются сочетанием клавиш Ctrl+Shift+B, а параметры запуска описаны в файле .vscode/launch.json.

Отладка выполняется отладчиком GDB. Точка останова устанавливается щелчком слева от строки или клавишей F9, запуск отладки –

клавишей F5. При остановке программы панель Run and Debug показывает стек вызовов Call Stack, значения локальных переменных Variables и Watch, а также консоль Debug Console. Команды Step Over (F10), Step Into (F11) и Step Out (Shift+F11) позволяют выполнять программу пошагово, а окно Variables показывает, как изменяются значения переменных.

2.4 Паттерн MVC

MVC (Model – View – Controller) – архитектурный паттерн, который разделяет приложение на три части. Модель хранит состояние и выполняет предметную логику, представление формирует интерфейс и отображает данные, контроллер обрабатывает действия пользователя и передаёт команды модели. Такое разделение уменьшает связанность классов: изменение интерфейса не требует изменения логики расчёта, а логику можно проверить независимо от графического интерфейса.

2.5 Активная и пассивная модель

В варианте с пассивной моделью контроллер после изменения данных сам инициирует обновление представления. Активная модель сама сообщает подписчикам об изменении своего состояния, то есть реализует паттерн «Наблюдатель» (Observer). В разработанном приложении класс CurrencyModel содержит список слушателей std::vector<Listener*> и после успешного расчёта вызывает у каждого из них метод onModelChanged(), поэтому главная форма обновляет метки автоматически.

Паттерн «Наблюдатель», на котором основана активная модель, реализован в приложении вручную. Интерфейс Listener объявляет единственный виртуальный метод onModelChanged(), а класс CurrencyModel хранит указатели на подписчиков в контейнере std::vector и вызывает этот метод у каждого из них после успешного изменения состояния.

Представление подписывается на модель в конструкторе главного окна и отписывается в деструкторе, поэтому модель никогда не обращается к удалённому объекту. Такая реализация не зависит от средств библиотеки Qt и может быть перенесена в любую программу на языке C++.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработано приложение для конвертирования белорусских рублей в евро. На главной форме в виде меток отображаются введённые числа и результат конвертации. По нажатию на кнопку «Ввод чисел» открывается дополнительное окно для ввода суммы и курса. Последние введённые данные запоминаются и восстанавливаются при повторном вводе. Если введены некорректные данные, выводится сообщение об ошибке. Архитектура приложения соответствует паттерну MVC с активной моделью.

Перевод выполняется по формуле:

результат, EUR = сумма, BYN / курс, BYN за 1 EUR

Например, 1500,50 BYN по курсу 3,2750 BYN за 1 EUR дают 458,17 EUR. В формулировке варианта упоминается «курс доллара»; поскольку пересчёт выполняется в евро, в окне ввода запрашивается курс евро (сколько белорусских рублей стоит один евро). Это уточнение согласовано с преподавателем.

3.2 Настройка среды и расширений для UML и SysML v2

Проект создан в Visual Studio Code в каталоге CurrencyConverter. Для работы с языком C++ установлен набор расширений C/C++ Extension Pack (IntelliSense, средства отладки, темы оформления), для сборки – расширение CMake Tools, которому указан набор инструментов «Qt 6.11.1 + MinGW 13.1 (x64)». В файле .vscode/settings.json заданы путь к библиотеке Qt 6.11.1 (CMAKE_PREFIX_PATH), компилятор C:/Qt/Tools/mingw1310_64/bin/g++.exe и генератор Ninja. Структура проекта показана на рисунке 3.1.

Файл CMakeLists.txt описывает цель сборки CurrencyConverter, подключает библиотеку Qt6::Widgets и содержит список исходных файлов проекта. Каталог build создаётся автоматически при настройке проекта и содержит исполняемый файл CurrencyConverter.exe, а также библиотеки Qt, скопированные рядом с ним. В файле .vscode/c_cpp_properties.json указаны пути к заголовочным файлам библиотеки, поэтому IntelliSense подсказывает имена классов и методов Qt, а панель Problems сразу показывает синтаксические ошибки.

Для работы с UML установлено расширение PlantUML (jebbs.plantuml), которое строит диаграммы из текстового описания и отображает их предпросмотр прямо в редакторе. Для работы с SysML v2 установлено расширение Syside Editor (sensmetry.syside-editor), обеспечивающее подсветку синтаксиса, проверку модели и навигацию по элементам языка SysML v2. Список установленных расширений приведён на рисунке 3.2.

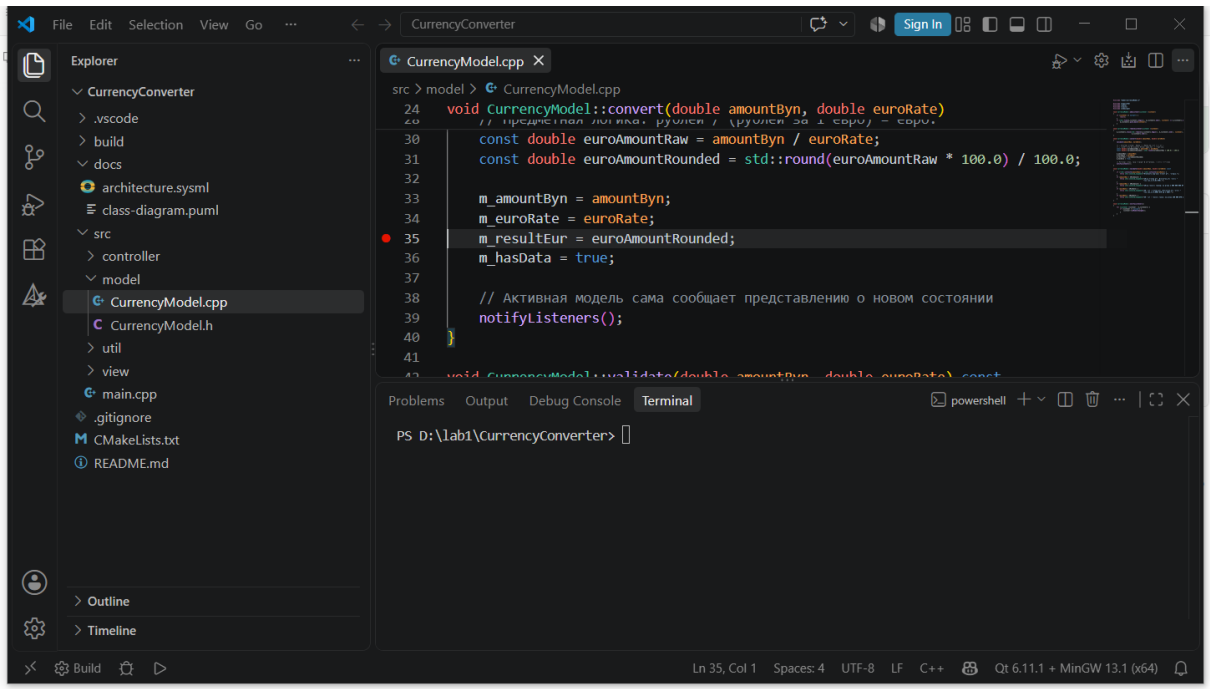


Рисунок 3.1 – Проект CurrencyConverter в Visual Studio Code

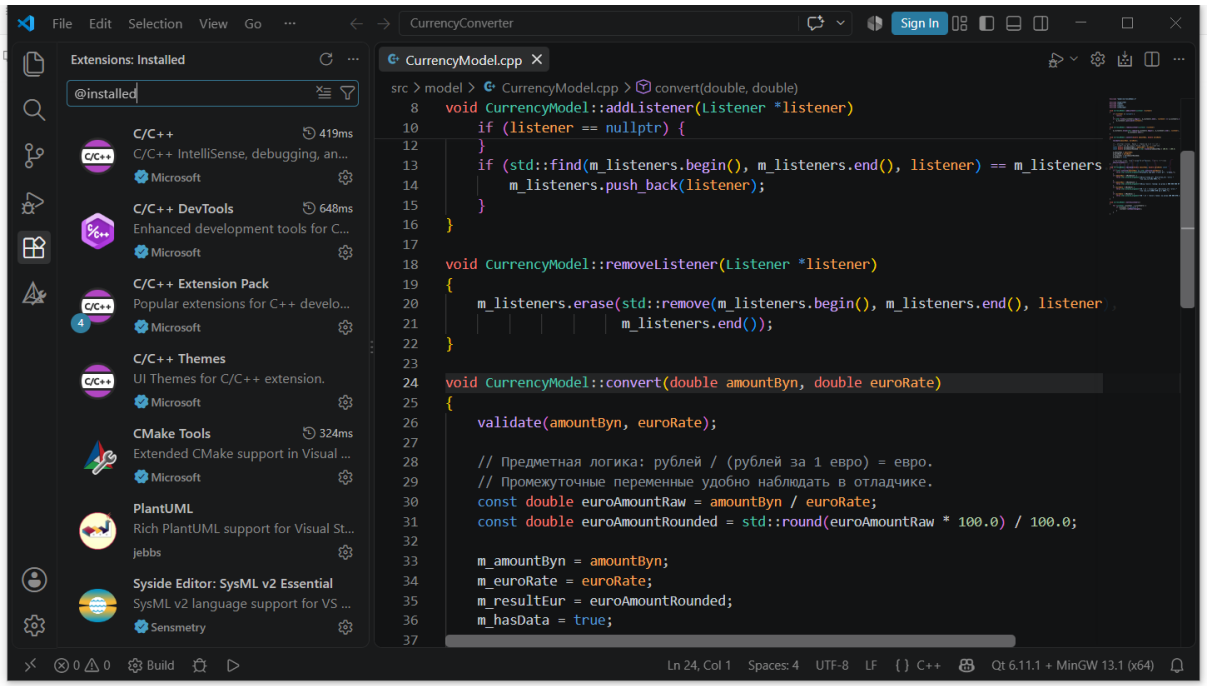


Рисунок 3.2 – Установленные расширения PlantUML и Syside Editor

Помимо сборки и отладки, при разработке использовались и другие возможности редактора: автодополнение IntelliSense с подсказками типов библиотеки Qt, переход к определению (F12) и поиск использований (Shift+F12), поиск по проекту (Ctrl+Shift+F), быстрые исправления (Ctrl+.) и панель Problems для контроля ошибок и предупреждений. На рисунке 3.3 показан результат поиска по проекту: строка notifyListeners найдена в четырёх местах трёх файлов проекта, переход по любому результату

выполняется одним щелчком.

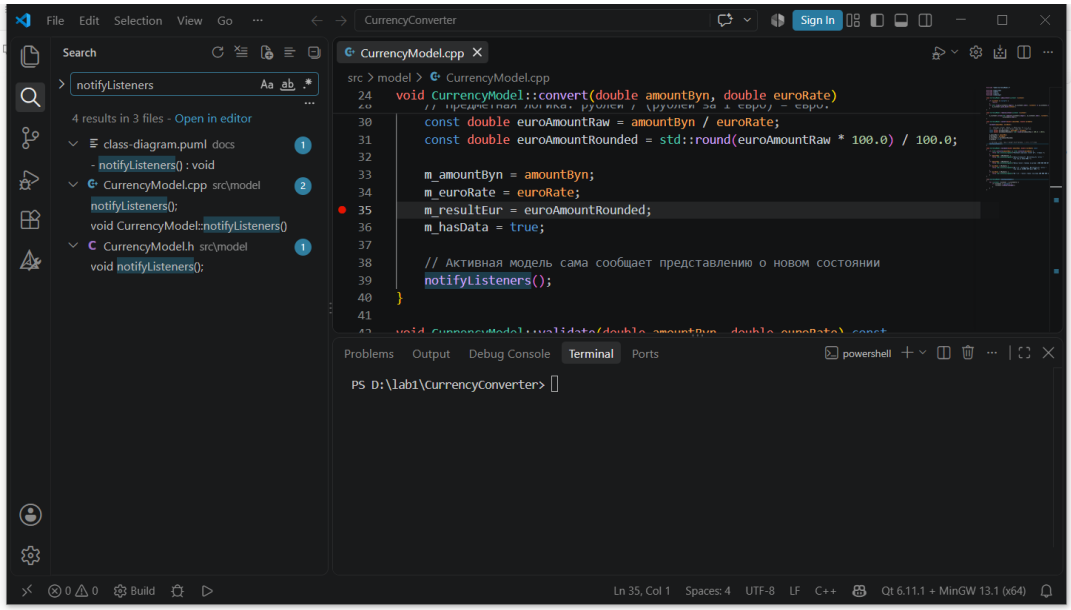


Рисунок 3.3 – Поиск по проекту в Visual Studio Code

Текстовая модель диаграммы классов находится в файле docs/class-diagram.puml. Предпросмотр диаграммы (команда PlantUML: Preview Current Diagram, сочетание клавиш Alt+D) показан на рисунке 3.4.

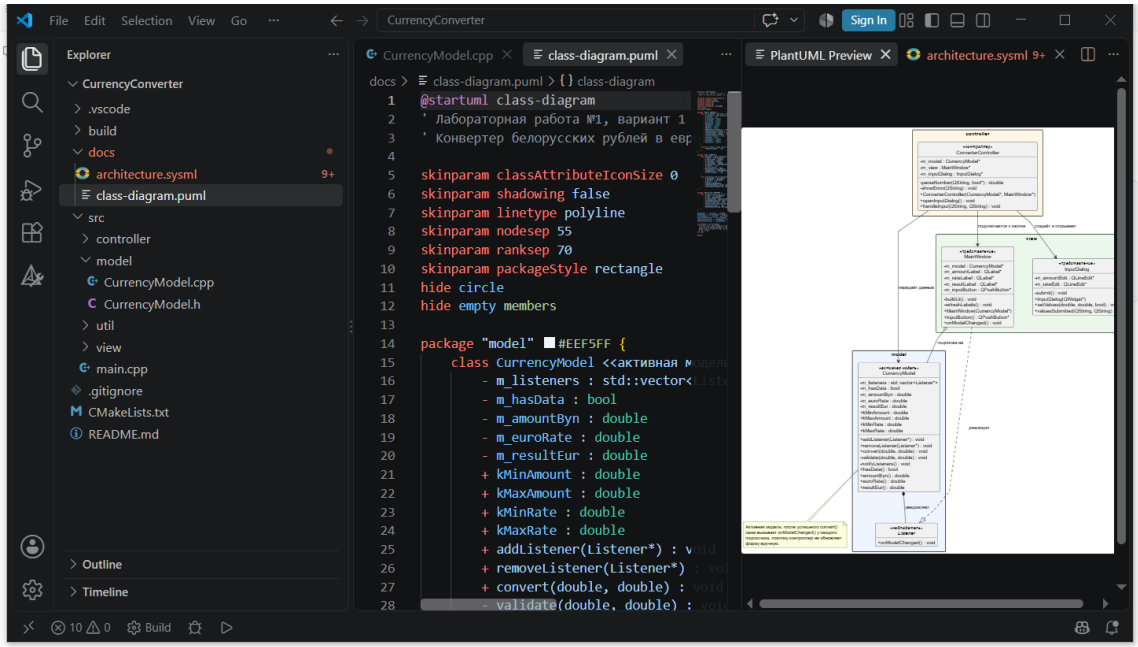


Рисунок 3.4 – Предпросмотр UML-диаграммы в Visual Studio Code

3.3 Демонстрация отладчика

Работа отладчика демонстрируется в методе CurrencyModel::convert. Точка останова устанавливается клавишей F9 на строке 35, в которой

вычисленное значение записывается в поле m_resultEur (рисунок 3.5).

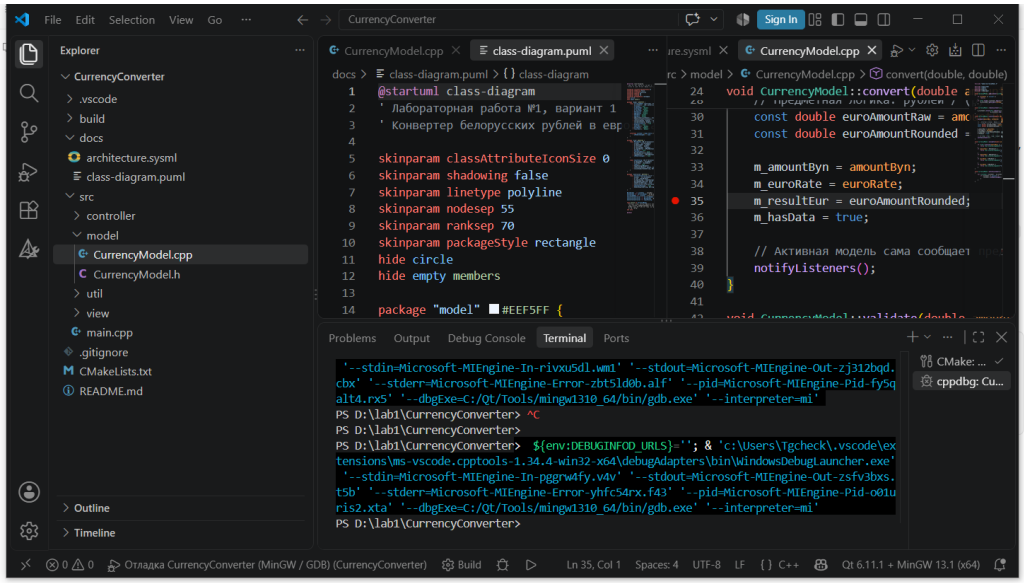


Рисунок 3.5 – Точка останова в методе CurrencyModel::convert

После запуска отладки (F5) в окне приложения вводятся сумма 1500,50 BYN и курс 3,2750 BYN за 1 EUR. При нажатии кнопки «ОК» программа останавливается на точке останова: панель Variables показывает локальные переменные euroAmountRaw = 458.16793893129773 и euroAmountRounded = 458.17000000000002, аргументы метода amountByn = 1500.5 и euroRate = 3.2749999999999999, а панель Call Stack показывает цепочку вызовов CurrencyModel::convert ← ConverterController::handleInput ← обработчик сигнала Qt (рисунок 3.6). Значения переменных подтверждают, что результат вычисляется корректно.

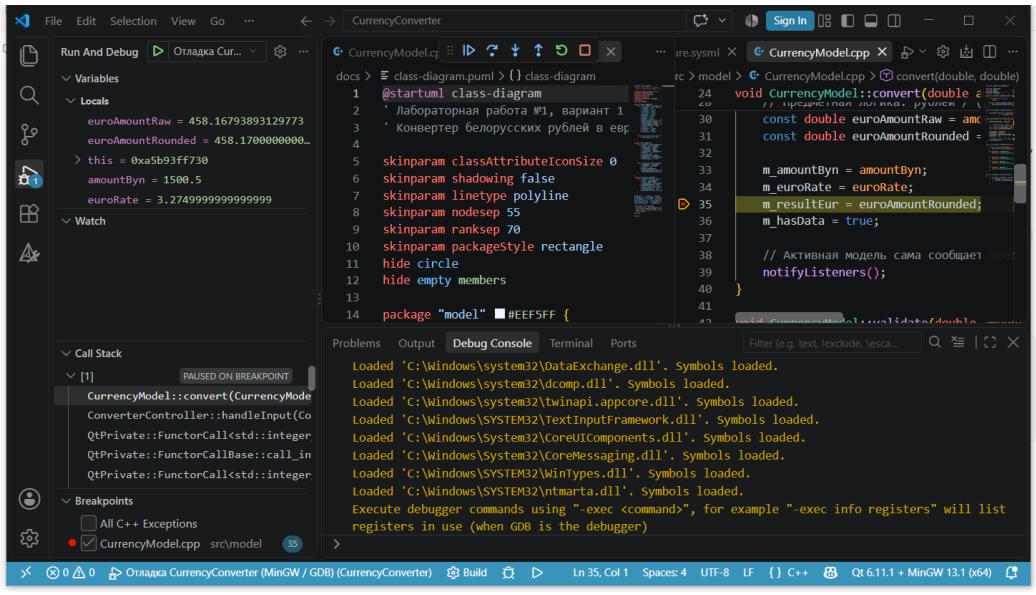


Рисунок 3.6 – Остановка программы на точке останова

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:

– CurrencyModel – модель: хранит сумму, курс и результат, выполняет проверку данных, расчёт и уведомление слушателей (активная модель);

– MainWindow – главная форма: метки введённых чисел и результата, реализует интерфейс CurrencyModel::Listener и обновляется автоматически;

– InputDialog – модальное окно ввода суммы и курса евро;

– ConverterController – контроллер: открывает окно ввода, преобразует текст в числа, передаёт данные модели и показывает сообщения об ошибках;

– main.cpp – точка входа, в которой собирается каркас MVC;

– Format.h – вспомогательные функции форматирования чисел для представления.

Каталоги проекта соответствуют слоям MVC: src/model, src/view, src/controller, src/util. Диаграмма классов приложения приведена на рисунке 3.7.

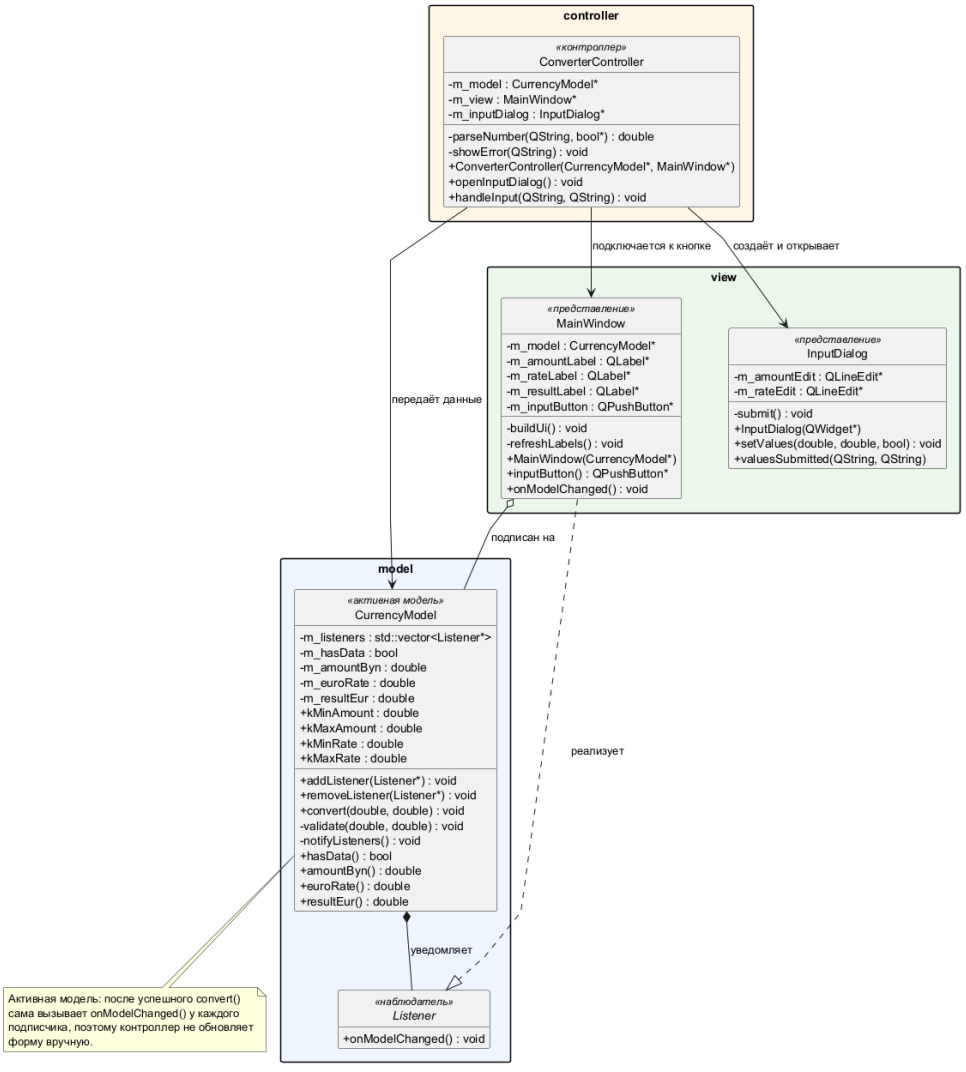


Рисунок 3.7 – Диаграмма классов приложения

3.5 Модель

Класс CurrencyModel реализует активную модель. Он хранит список подписчиков std::vector<Listener*> и предоставляет методы addListener() и removeListener(). Интерфейс Listener объявляет единственный метод onModelChanged(), который реализует представление.

Метод convert() выполняет многоуровневую проверку входных данных: значения должны быть числами, сумма – находиться в диапазоне от 0,01 до 1 000 000 000 BYN, курс – в диапазоне от 0,0001 до 100 000 BYN за 1 EUR. При нарушении ограничений метод возбуждает исключение std::invalid_argument с текстом, понятным пользователю. После проверки вычисляется результат, а затем вызывается notifyListeners(), который уведомляет всех подписчиков. Именно этот вызов делает модель активной: контроллер не обновляет форму вручную.

3.6 Представление

Класс MainWindow отображает последние введённые числа и результат конвертации. Благодаря реализации интерфейса CurrencyModel::Listener метод onModelChanged() обновляет метки сразу после того, как модель зафиксировала изменения. До первого расчёта метки содержат прочерки, поэтому форма корректно выглядит и при запуске.

Класс InputDialog обеспечивает ввод суммы и курса. Окно не содержит бизнес-логики: при нажатии кнопки «ОК» или клавиши Enter оно передаёт введённый текст контроллеру сигналом valuesSubmitted(), а метод setValues() подставляет последние успешно введённые значения при повторном открытии окна.

3.7 Контроллер

Класс ConverterController управляет жизненным циклом окна ввода и отвечает за безопасное преобразование пользовательского текста в числа. Метод parseNumber() допускает разделение дробной части запятой или точкой и игнорирует пробелы, используемые как разделители тысяч. Метод openInputDialog() подключает обработчик сигнала кнопки «Ввод чисел», создаёт окно ввода один раз и перед показом восстанавливает в нём последние значения из модели. Метод handleInput() преобразует строки в числа, передаёт их модели и при возникновении исключения показывает пользователю окно с сообщением об ошибке, не закрывая окно ввода.

3.8 Запоминание данных и обработка ошибок

Последние успешно обработанные значения хранятся в модели, поэтому при повторном нажатии кнопки «Ввод чисел» окно ввода открывается с уже заполненными полями. Если пользователь ввёл

нечисловое значение, отрицательную сумму или нулевой курс, контроллер перехватывает исключение и выводит сообщение об ошибке, а окно ввода остаётся открытым для исправления данных. Главная форма при этом сохраняет предыдущий корректный результат.

3.9 Модель SysML v2

Структурное описание приложения подготовлено в текстовой нотации SysML v2 и хранится в файле docs/architecture.sysml. Модель содержит определения частей (part def) CurrencyModel, MainWindow, InputDialog, ConverterController и Application, а также порты UserInputPort и ModelChangedPort и соединения между ними, которые описывают поток управления «пользователь – контроллер – модель» и обратный поток уведомлений активной модели. Файл открыт в Visual Studio Code на рисунке 3.8; расширение Syside Editor выполняет проверку модели – панель Problems не содержит замечаний.

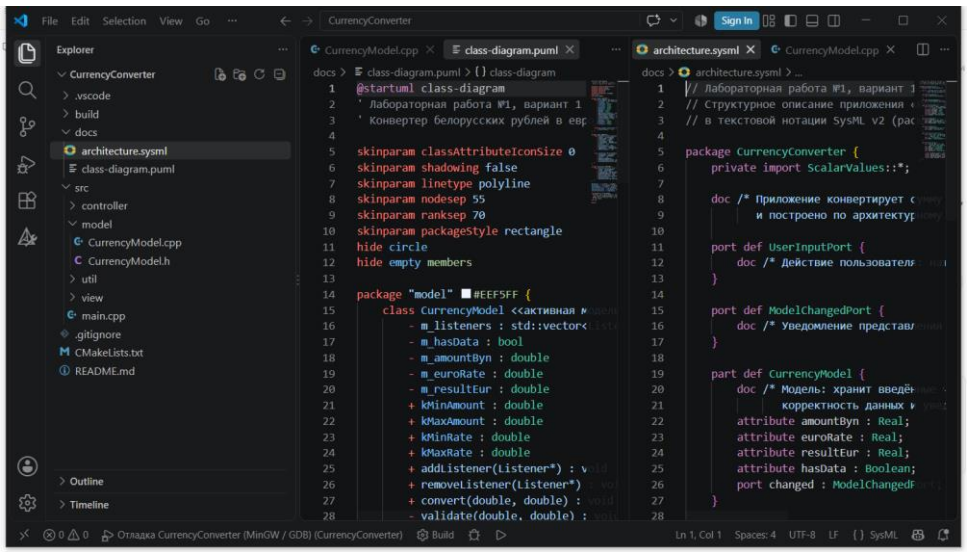


Рисунок 3.8 – Структурная модель приложения на языке SysML v2

4 РЕЗУЛЬТАТЫ РАБОТЫ

В Visual Studio Code создано и проверено приложение варианта 1 на языке C++ с библиотекой Qt 6. Главная форма после запуска показана на рисунке 4.1: метки содержат прочерки, так как расчёт ещё не выполнялся.

Проверены следующие сценарии работы приложения: корректный ввод суммы и курса с расчётом результата, повторное открытие окна ввода с восстановлением последних значений, а также три вида ошибочного ввода – нечисловая сумма, отрицательная сумма и нулевой курс. Во всех случаях приложение ведёт себя предсказуемо: результат выводится на главную форму, а сообщение об ошибке не приводит к потере ранее введённых данных.

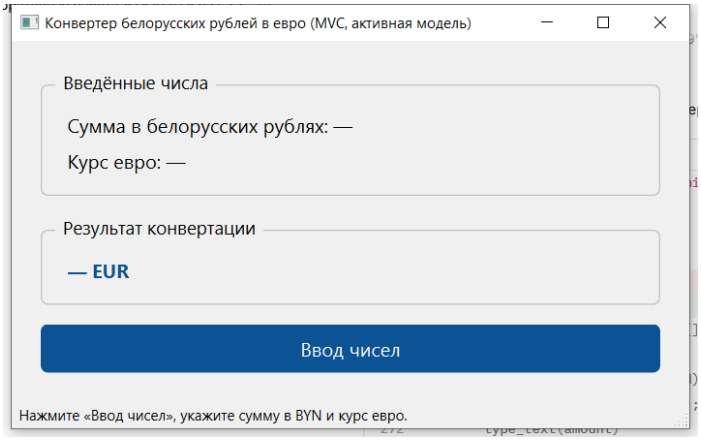


Рисунок 4.1 – Главная форма после запуска

По нажатию на кнопку «Ввод чисел» открывается модальное окно ввода (рисунок 4.2). Поля снабжены подсказками, а дробную часть можно отделять запятой или точкой.

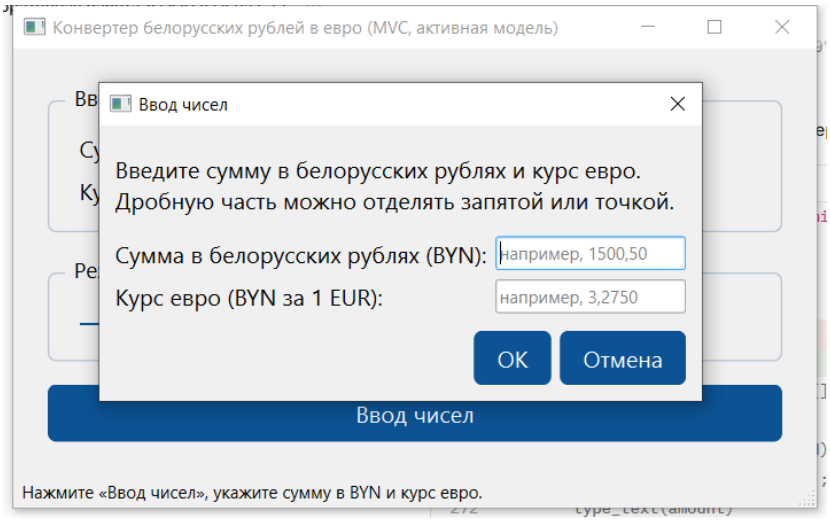


Рисунок 4.2 – Окно ввода данных

На рисунке 4.3 показан ввод суммы 1500,50 BYN и курса 3,2750 BYN за 1 EUR, на рисунке 4.4 – результат конвертации 458,17 EUR на главной форме.

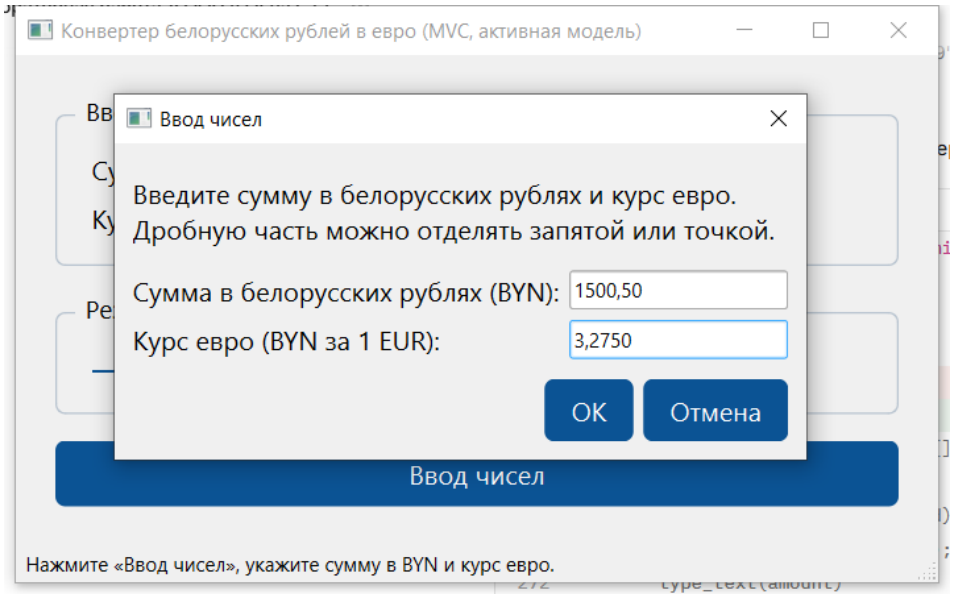


Рисунок 4.3 – Ввод суммы и курса евро

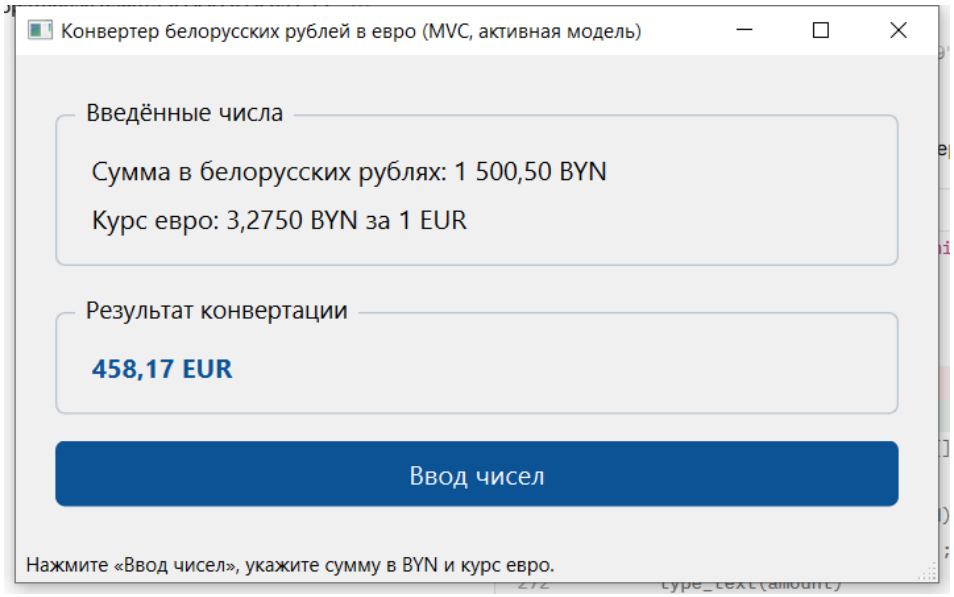


Рисунок 4.4 – Результат конвертации на главной форме

Повторное открытие окна ввода подтверждает запоминание данных: поля заполнены последними успешно введёнными значениями (рисунок 4.5).

Сохранение данных выполняет модель: после успешного расчёта она хранит сумму и курс, а контроллер при открытии окна передаёт их в поля ввода. Если пользователь закрывает окно кнопкой «Отмена», последние

корректные значения сохраняются и будут предложены при следующем открытии окна.

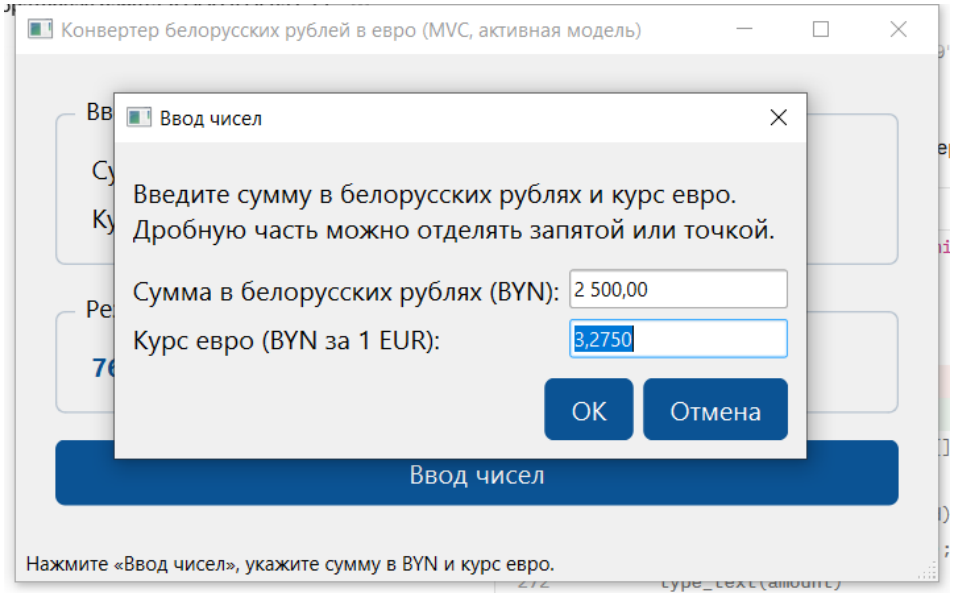


Рисунок 4.5 – Восстановление последних введённых данных

Проверены три сценария ошибочного ввода. При нечисловой сумме выводится сообщение «Сумма «abc» не является числом» (рисунок 4.6), при отрицательной сумме – «Сумма должна быть положительным числом (не меньше 0,01 BYN)» (рисунок 4.7), при нулевом курсе – «Курс евро должен быть положительным числом (не меньше 0,0001 BYN за 1 EUR)» (рисунок 4.8). Во всех случаях окно ввода остаётся открытым, а главная форма сохраняет предыдущий корректный результат.

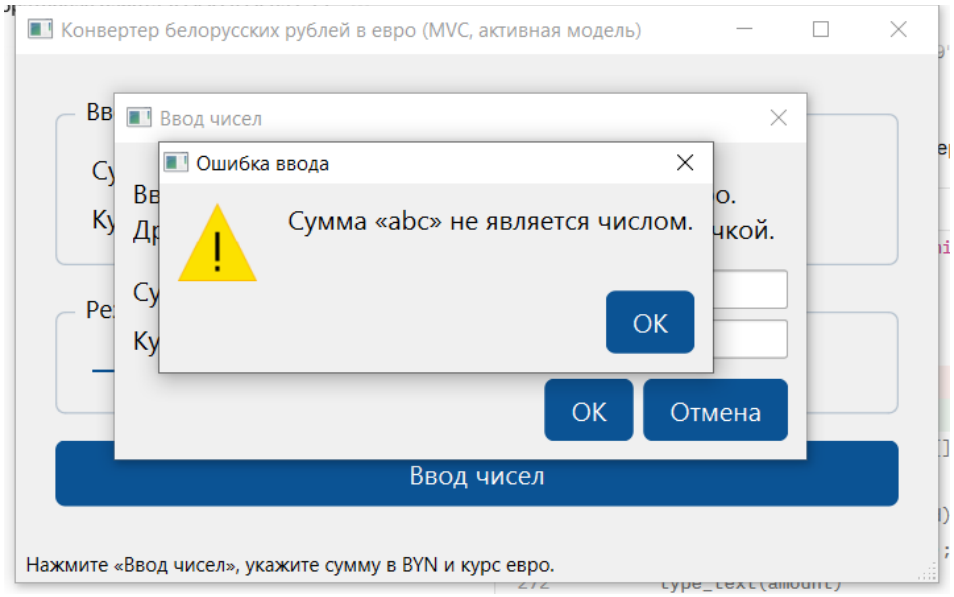


Рисунок 4.6 – Сообщение об ошибке при нечисловой сумме

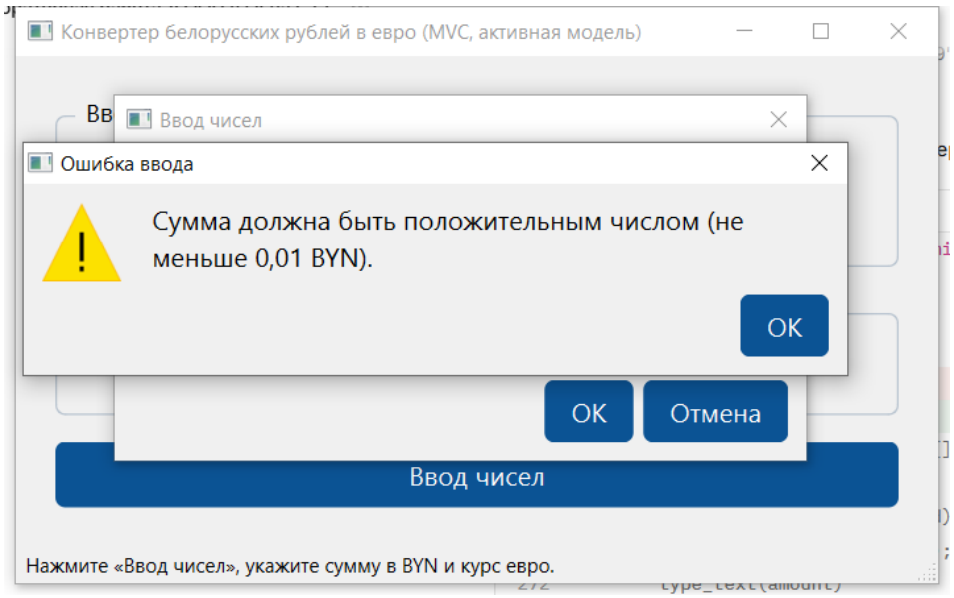


Рисунок 4.7 – Сообщение об ошибке при отрицательной сумме

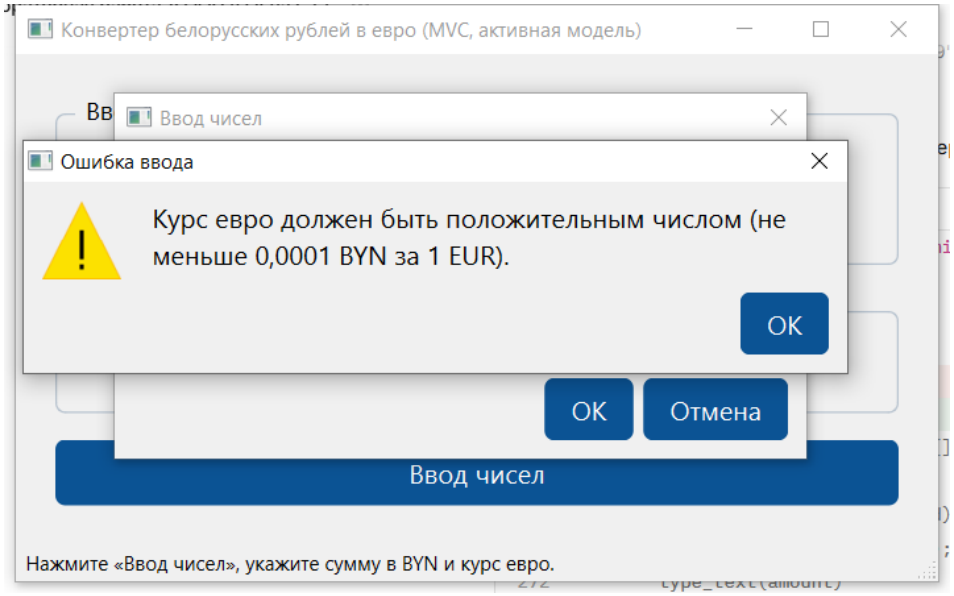


Рисунок 4.8 – Сообщение об ошибке при нулевом курсе

После исправления данных конвертация выполняется успешно: 5 000,00 BYN по курсу 3,2750 BYN за 1 EUR дают 1 526,72 EUR (рисунок 4.9). Архитектура приложения соответствует MVC с активной моделью: контроллер обрабатывает действие пользователя, модель выполняет проверку и расчёт, а затем сама уведомляет представление через интерфейс Listener.

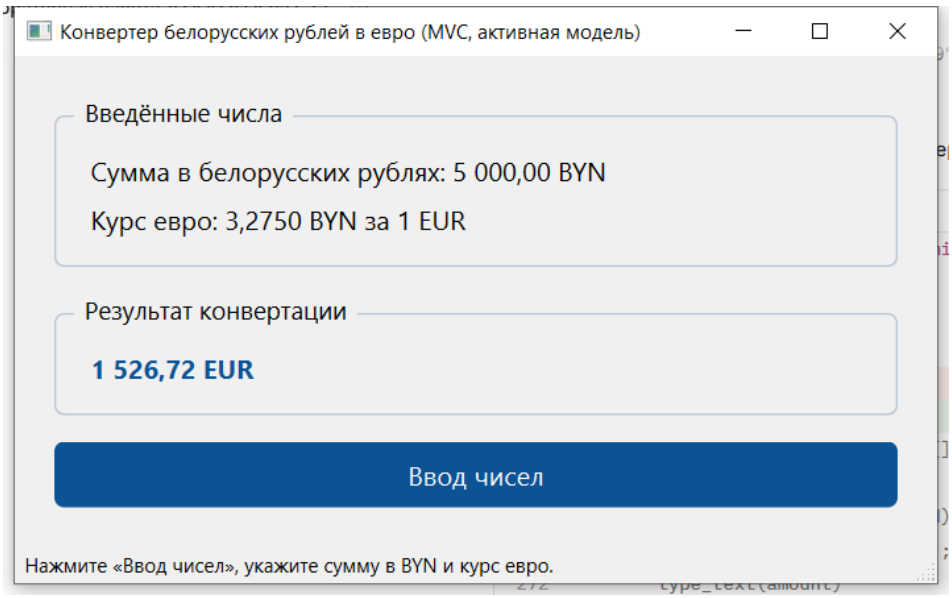


Рисунок 4.9 – Итоговый результат конвертации

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Какие компоненты входят в состав IDE?

В состав интегрированной среды разработки входят редактор исходного кода, компилятор или интерпретатор, система сборки и запуска, отладчик, средства навигации и рефакторинга, терминал, интеграция с системой контроля версий и механизм расширений. В Visual Studio Code перечисленным компонентам соответствуют редактор с IntelliSense, задачи сборки (tasks.json) и CMake Tools, отладчик GDB, панели Explorer, Problems, Outline, терминал, панель Source Control и магазин расширений Marketplace.

Большинство перечисленных компонентов в Visual Studio Code подключены через расширения: например, поддержка языка C++ и средства отладки появляются после установки набора C/C++ Extension Pack, а сборка выполняется расширением CMake Tools. Такое построение позволяет использовать одну среду для разных языков и технологий.

Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся автодополнение и подсказки типов (IntelliSense), быстрые исправления, автоматическое форматирование, переход к определению и поиск использований, безопасное переименование, поиск по проекту, работа с несколькими курсорами и анализ предупреждений в панели Problems. В Visual Studio Code это сочетания клавиш Ctrl+P, Ctrl+Shift+P, F12, Shift+F12, F2, Ctrl+Shift+F и Shift+Alt+F. Эти приёмы ускоряют изменение кода и уменьшают число механических ошибок.

Навигацию по проекту упрощают сочетания Ctrl+P (переход к файлу по имени) и Ctrl+Shift+F (поиск по всем файлам), а переход к определению (F12) и поиск использований (Shift+F12) позволяют быстро разобраться в незнакомом коде. Автоматическое форматирование поддерживает единый стиль, а панель Problems заранее предупреждает о потенциальных ошибках.

Каким образом выполняется сборка и запуск проекта?

Проект описан файлом CMakeLists.txt. Настройка сборки выполняется командой cmake -S . -B build -G Ninja с указанием пути к библиотеке Qt 6.11.1, компилятора MinGW g++ 13.1 и генератора Ninja, а сборка – командой cmake --build build. В Visual Studio Code эти команды оформлены задачей, которая запускается сочетанием клавиш Ctrl+Shift+B. Запуск приложения выполняется файлом launch.json: клавиша F5 запускает программу под управлением отладчика GDB, сочетание Ctrl+F5 – без отладчика. Готовый исполняемый файл CurrencyConverter.exe находится в каталоге build.

Задачи сборки описаны в файле tasks.json, а параметры запуска и отладки – в файле launch.json. Настройка проекта командой cmake

выполняется один раз: создаётся каталог build с файлами системы сборки, после чего повторные сборки занимают несколько секунд. Перед запуском отладчика сборка выполняется автоматически.

Что такое MVC?

MVC – архитектурный паттерн, который разделяет данные и предметную логику, графический интерфейс и обработку действий пользователя. В данной работе роль модели выполняет класс CurrencyModel, роль представления – классы MainWindow и InputDialog, роль контроллера – класс ConverterController. Такое разделение позволяет изменять интерфейс независимо от логики расчёта и упрощает проверку программы.

Разделение на три компонента уменьшает связанность кода: изменение интерфейса не затрагивает логику расчёта, а проверку данных можно выполнять независимо от графического интерфейса. В приложении это позволило описать все ограничения и сообщения об ошибках в модели, а обработку действий пользователя – в контроллере.

Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление, поэтому после её изменения контроллер должен сам обновить интерфейс. Активная модель публикует событие при изменении своего состояния. В приложении CurrencyModel хранит список подписчиков и после успешного расчёта вызывает метод onModelChanged() у каждого из них, а MainWindow, реализуя интерфейс CurrencyModel::Listener, самостоятельно обновляет метки формы. Это исключает дублирование кода обновления интерфейса в контроллере.

Активная модель удобна, когда состояние изменяется из нескольких мест: каждое изменение автоматически отражается в интерфейсе. Пассивная модель проще в реализации, но требует, чтобы разработчик не забывал обновлять форму после каждого изменения данных. В данной работе выбрана активная модель, поэтому главная форма обновляется автоматически после каждого успешного расчёта.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности Visual Studio Code для редактирования, сборки, запуска и отладки приложения на языке C++. Рассмотрены средства работы с UML и SysML v2: установлены расширения PlantUML и Syside Editor, подготовлена диаграмма классов и структурная модель приложения.

Для варианта 1 разработано приложение конвертирования белорусских рублей в евро с графическим интерфейсом на библиотеке Qt 6. Приложение отображает введённые числа и результат в виде меток, открывает отдельное окно ввода суммы и курса, восстанавливает последние введённые данные и выводит понятные сообщения об ошибках при некорректном вводе. Компоненты разделены по паттерну MVC, а активная модель уведомляет представление об изменении состояния через интерфейс Listener.

В ходе работы были освоены основные приёмы работы в Visual Studio Code: навигация по проекту, автодополнение IntelliSense, сборка через CMake и отладка программы с помощью GDB.

Разработанное приложение выполняет все требования варианта 1: конвертирует белорусские рубли в евро, отображает введённые числа и результат в виде меток, запоминает последние введённые данные, восстанавливает их при повторном открытии окна и сообщает об ошибках ввода. Проверка подтвердила правильность расчётов: 1500,50 BYN по курсу 3,2750 BYN за 1 EUR дают 458,17 EUR.

Исходный код разделён на модель, представление и контроллер. Активная модель уведомляет представление об изменениях, поэтому добавление новых полей не потребует переработки кода обновления интерфейса.